Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 00 (2026) 000–000

Procedia Computer Science

www.elsevier.com/locate/procedia

The 9th International Conference on Emerging Data and Industry (EDI40)
April 14-16, 2026, Istanbul, Türkiye

A Hallucination-Resistant RAG System for Software Update Related Prompts

Farheen Shaikh [a], Solomon Berhe* [b][1]

University of the Pacific, Stockton, 95204 California, USA
University of the Pacific, Stockton, 95204 California, USA

Abstract

Release notes for operating systems, applications, and software in Internet of Things (IoT) ecosystems evolve rapidly across fragmented sources, making it difficult for users and organizations to stay informed. To address these challenges, modern tools such as large language models (LLMs) and retrieval-augmented generation (RAG) offer new opportunities for automating update summarization and information access. However, although LLMs can effectively summarize updates, they frequently hallucinate when evidence is incomplete, stale, or outside expected query patterns. In this domain, a hallucinated answer is not merely a small accuracy error, it can lead to unsafe operational update decisions, such as misidentifying critical security patches, deploying the wrong software version, or overlooking vendor advisories. We present ReleaseGuard, a gated retrieval-augmented system that reduces hallucination by implementing query-intent filtering and evidence-aware abstention. Instead of generating answers by default, the system evaluates the relevance, coverage, and freshness of retrieved evidence before allowing the model to respond. When support is insufficient, ReleaseGuard outputs an explicit "I don't know" response, avoiding fabricated update information. This abstention-first design improves response quality by ensuring that generated answers are grounded, verifiable, and backed by adequate evidence. Overall, ReleaseGuard provides a more accurate, dependable, and safety-oriented approach towards software-update decisions.

Keywords: Retrieval-Augmented Generation (RAG); Abstention; Hallucination Mitigation; Software Update Intelligence; Release Notes; Evidence-Aware Reasoning; Query-Intent Filtering; AI Safety; Live Data Integration; LLM Reliability; Grounded Generation; Risk-Aware AI Systems.

[1]* Corresponding author. Tel.: +1-772-202-3743
E-mail address: sberhe@pacific.edu

1877-0509 © 2026 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the Conference Program Chairs.

1. Introduction

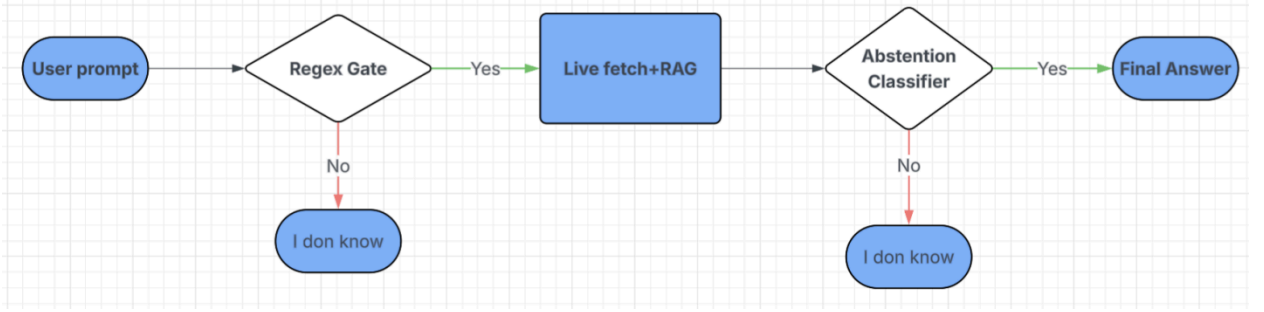
Large Language Models (LLMs) are increasingly used as natural-language interfaces for technical information access. However, they remain prone to hallucination, often producing fluent but incorrect statements when evidence is incomplete, outdated, or ambiguous. This challenge is particularly severe in IoT software-update related decisions, where accuracy directly impacts operational safety and user decision-making. In these IoT environments, release notes are updated in near-realtime. Examples include operating systems, databases, libraries, and user facing applications. Dependent release notes are often distributed across heterogeneous sources such as vendor feeds, community forums, changelogs, GIT commits, and security advisories.

When an LLM is queried about a software update without sufficient or fresh evidence, it may fabricate version numbers, invent changelog items, or provide misleading patch details. Such errors can propagate into user decisions and downstream software release processes, creating risks for misapplied patches, failed deployments, and operational instability. Prior studies show that retrieval-augmented systems, including GitHub Copilot, GitBook AI, and Atlassian's Release Notes AI, continue to hallucinate even when retrieval results are weak or stale. Moreover, empirical evaluations further demonstrate that tools like GitHub Copilot may generate incorrect or fabricated API calls despite the availability of relevant documentation, reflecting broader limitations of retrieval-based grounding.

Although research in honesty alignment and abstention encourages LLMs to answer "I don't know" when uncertain, existing systems rarely incorporate domain-specific signals such as evidence freshness, cross-source consistency, or structured intent patterns 3. As a result, generation remains the default behavior, leaving update assistants vulnerable to unsupported or speculative claims. These limitations underscore the need for mechanisms that prevent LLMs from producing fabricated or outdated software-update details. The operational consequences of incorrect update guidance,including system outages, dependency breaks, and patch failures,have been documented in prior software-engineering studies (cite 4).

This paper addresses a foundational question: *How can an LLM-based update assistant determine not only how to answer, but whether it should answer at all?* We argue that software-update decision support must be reframed as an answerability assessment problem in which abstention is a principled, pre-generation decision rather than a reactive safeguard. This perspective motivates **ReleaseGuard**, a system that evaluates the strength, coverage, and freshness of retrieved evidence before allowing any answer to be generated. By conditioning generation on evidence sufficiency, ReleaseGuard aims to improve stable, trustworthy, and verifiable update reasoning.

Figure 1: High-level decision flow of ReleaseGuard showing how queries are gated, evidence signals are computed, and abstention occurs before any generation step



2. Related Work

Retrieval-augmented generation (RAG) systems have been widely explored as a mechanism for grounding LLM outputs in external evidence. Early systems such as REALM, ATLAS, and RETRO assume that retrieved documents provide sufficient contextual support for generation. However, several studies show that this assumption frequently fails: when retrieval is incomplete, noisy, or temporally outdated, RAG models continue to hallucinate despite having access to external text. RARR reframes RAG as a rewriting procedure to mitigate unsupported content, while FreshPrompt demonstrates that LLMs systematically over-trust stale documents, highlighting the need for recency-aware retrieval in dynamic domains such as software updates.

A complementary line of work focuses on post-generation verification. Frameworks such as LLM Verification and Program-of-Thought Verification evaluate and filter model outputs after they are produced, often improving correctness by identifying inconsistencies or unsupported reasoning steps. These methods, however, operate reactively: they still require the model to generate a candidate answer before determining whether that answer is reliable. In contrast, ReleaseGuard treats *answerability* as a pre-generation decision, aiming to prevent unsupported responses from being produced at all.

Safety-aligned models offer another perspective. Techniques such as Constitutional AI and SCAlign enforce restricted or ethical behaviors through refusal policies, but they do not incorporate domain-specific signals such as evidence freshness, cross-source agreement, or update-timeline volatility. As a result, they remain vulnerable to hallucinations in settings where factual accuracy depends on rapidly changing information rather than moral or behavioral constraints. ReleaseGuard complements this line of work by applying refusal not for safety reasons, but for evidence insufficiency.

Recent work in automated program repair (APR) introduces the concept of *bug abstention*, where a model predicts whether a repair attempt is likely to succeed and abstains when confidence is low. This reduces wasted developer effort and improves trust in automated repair systems. Although focused on a different domain, this approach demonstrates the broader value of abstention-based decision pipelines for improving the reliability of AI systems. ReleaseGuard adopts a similar philosophy for software-update question answering, determining *whether* a query is answerable before initiating generation.

A separate and unexpected research direction shows that hallucination can sometimes improve performance in controlled reasoning tasks. The null-shot prompting framework demonstrates that intentionally inserting hallucinated information into a prompt can improve arithmetic reasoning accuracy by up to 44%. While this reveals complex dynamics in LLM behavior, such mechanisms are incompatible with safety-critical applications. In domains like software-update decision support, where fabricated version numbers or imaginary patch information can lead to deployment failures, misconfigured systems, or operational downtime,hallucination must be minimized rather than exploited.

Across RAG grounding, verification, safety alignment, and abstention frameworks, a shared structural limitation persists: generation remains the default action, and refusal occurs only after a potential failure. Existing systems rarely incorporate domain-aware metadata such as evidence recency, multi-source consistency, or the presence of identifiable vendor signals. ReleaseGuard addresses this gap by introducing a pre-generation abstention mechanism that evaluates evidence strength, reliability, and freshness before allowing an LLM to answer. This abstention-first design shifts hallucination mitigation from a reactive correction process to a principled determination of answerability, making it particularly suitable for fast-evolving and operationally sensitive software-update environments.

Table 1: Comparison of ReleaseGuard with Prior Work

Dimension	RAG Systems (REALM, ATLAS, RARR, FreshPrompt)	Abstention Models (AbstainQA, IDK prompting)	Safety / Refusal Control (Constitutional AI, SCAlign)	ReleaseGuard (This Paper)
Goal	Ground answers via retrieval	Avoid incorrect answers via uncertainty or refusal	Block unsafe or undesired behaviors	Answer only when evidence is sufficient, relevant, and fresh
Core Assumption	Retrieved docs ⇒ enough evidence	Confidence ≈ correctness	Prompts can enforce safety	Evidence must be present, reliable, and up-to-date
Outdated Evidence	Often ignored (FreshPrompt shows failures)	Ignored in most models	Ignored	Explicit freshness threshold and cross-source checks
Response Strategy	Always answer, unless retrieval fails	Answer or refuse after generation	Refusal allowed but not domain-aware	Regex gate + freshness gate + abstention-first decision
Domain Signals Used	Similarity scores only	None (model-centric)	None (prompt-centric)	Vendor names, product tokens, timestamps, cross-source consistency
Typical Failure Mode	Hallucinates with weak or stale retrieval	Refuses too late (post-generation)	Safe but not hallucination-aware	Blocks unsupported answers before generation
Live + Historical Fusion	Rare (RAGTruth shows issues)	Not supported	Not supported	Full hybrid RAG + live API fusion
Safety Stage	During or after generation	Post-generation verification	Pre-query behavioral safety	Pre-generation abstention + evidence sufficiency test

3. Method

This section describes the construction of the evaluation setting, the abstention-first architecture of ReleaseGuard, and the comparison against a baseline RAG system. The overall goal is to determine whether an update-oriented LLM system can identify *when not to answer* based on evidence of sufficiency, freshness, and reliability.

3.1 Vendor Catalogue Dataset

ReleaseGuard makes use of a vendor catalogue containing the names of software products commonly referenced in update-related queries. Although the catalogue does not include version information, it plays two essential roles. First, it defines the set of entities for which update information may be requested. Second, it provides a controlled vocabulary that facilitates consistent matching between user queries and evidence retrieved from live software-update feeds. The catalogue therefore serves as a domain lexicon supporting intent parsing, token normalization, and the filtering of potentially relevant evidence across both well-known and long-tail products.

3.2 Query Construction

To evaluate hallucination control, an evaluation set is derived from the vendor catalogue. Queries are grouped into three categories. The first consists of well-known products with reliable update traces (“head vendors”). The second comprises long-tail products with sparse or incomplete release histories (“tail vendors”). The third introduces noisy variants formed through misspellings or partial names that do not match catalogue entries exactly. This design tests the system’s ability to distinguish between answerable queries and queries for which evidence is insufficient, ambiguous, or outdated.

3.3 System Architecture

ReleaseGuard is formulated as an abstention-first update question-answering framework. The pipeline consists of five components:

$$\mathcal{R} = (Q, G, R, D, \Gamma).$$

(1) Q a space of user queries; an intent-gating module that filters for well-formed, update-focused requests

(2) G a query gate that restrict user to standard prompt template
(3) R a retrieval module that collects potentially relevant evidence from software-update sources
(4) D an abstention module that determines whether the retrieved evidence is adequate to support an answer
(5) Γ a constrained generator that is invoked only if the abstention module authorizes generation

This structure parallels earlier retrieval and verification pipelines, but differs in one critical respect: generation is not the default action. Instead, ReleaseGuard evaluates answerability *before* engaging any generative model. The decision pipeline is illustrated in Fig. 2.

3.4 Evidence Signal

ReleaseGuard evaluates each query using three domain-specific evidence signals. The first is a relevance signal, which measures how closely the retrieved context aligns with the user's query (sMax). The second is a coverage signal that reflects the amount of live evidence available for the IoT software product being queried (h). The third is a freshness signal that captures the recency of the most recent update information associated with the product (delta). Together, these signals provide a structured indication of whether the system has sufficient, reliable, and up-to-date evidence to support an answer.

For example, a query (q) such as "latest React version" typically yields evidence from sources like recent NPM releases or active developer discussions, producing high relevance and coverage scores with low staleness. In contrast, less common plugins or long-tail software products often produce no relevant hits and only stale references, indicating insufficient evidence. ReleaseGuard uses this combined signal profile to judge whether generating an answer is appropriate or whether abstention is required.

We represent these signals as:

$$\phi(q) = (s_{\max},\ h,\ \Delta).$$

3.5 Abstention Decision Rule

ReleaseGuard assigns a decision D(q) based on the evidence vector.

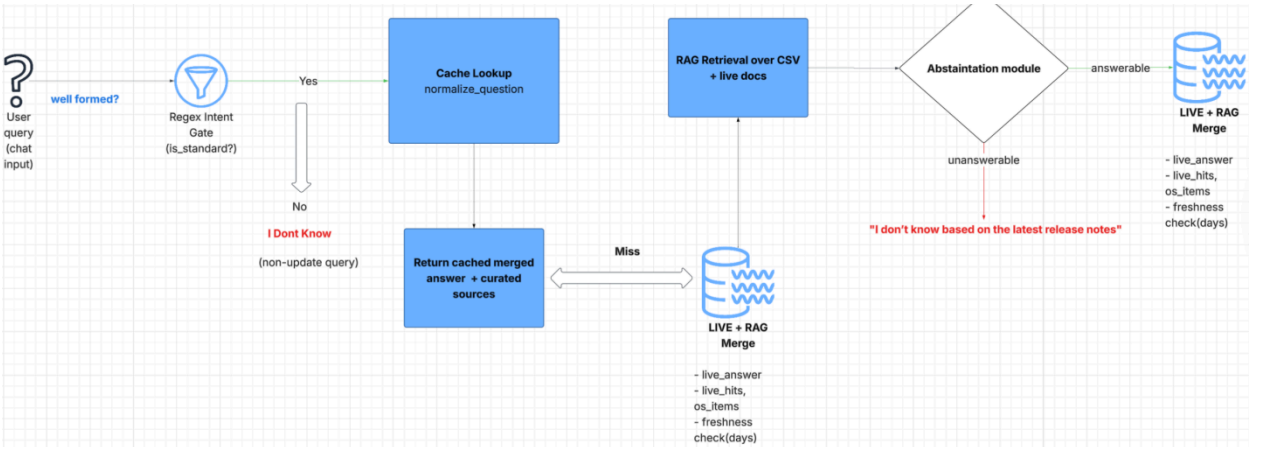
The system **abstains** when the maximum similarity score is below the relevance threshold and there is **no** live evidence available. It also abstains when the query explicitly requests the "latest" version but the most recent evidence is **older** than the freshness threshold . In all other cases, where evidence is sufficiently relevant, supported, and fresh, the system proceeds to **answer** the query.

$$D(q) = \begin{cases} \text{Abstain}, & s_{\max} < \tau_s \wedge h = 0, \\ \text{Abstain}, & \text{the query requests “latest” and } \Delta > \tau_t, \\ \text{Answer}, & \text{otherwise.} \end{cases}$$

3.6 Operational Pipeline

The operational pipeline of ReleaseGuard proceeds through a structured sequence of decision stages as shown in Figure 2. A user query first enters the intent gate G(q), which filters out prompts that do not match update-related patterns. For queries that pass this stage, the retrieval module R gathers candidate evidence from both historical sources and live update feeds. From this retrieved context, the system computes the evidence signals, representing relevance, coverage, and freshness. These signals are then evaluated by the abstention classifier, which determines whether sufficient, up-to-date, and reliable evidence exists to justify generating an answer. If the evidence satisfies all thresholds, the generator Γ produces a grounded response; otherwise, the system halts early and returns an abstention. This pipeline ensures that weak, missing, or stale evidence results in refusal rather than unsupported or hallucinated output.

Figure 2: architecture of ReleaseGuard as an abstention-first update–question answering system R = (Q, G R, D, Γ). Queries are intent-gated by G, evidence is retrieved by R from historical and live sources, and the abstention module Γ decides whether to invoke the constrained generator Γ or return “I don’t know.”



3.7 System Compared

Two systems are compared using identical data sources, retrieval infrastructure, and LLM. The baseline RAG system retrieves documents and always produces an answer regardless of evidence sufficiency. ReleaseGuard follows the abstention-first pipeline, incorporating a regex-based intent gate, caching via Redis, live update retrieval from OS and community feeds, hybrid FAISS–BM25 retrieval, and evidence-based abstention before generation.

3.8 Evaluate Metrics

For evaluating system behavior, each output is manually annotated using three mutually exclusive labels:

- **Answer** – The system provides a response that can be verified using authoritative vendor sources (official websites, release feeds, or documentation).

- **Abstain** – The system explicitly refuses to answer and gives a justification indicating insufficient or unreliable evidence.
- **Hallucination** – The system outputs a version number, date, or release detail that cannot be confirmed by any trusted source.

From these annotations, we compute the following proportional metrics:

- **Answer rate** – The fraction of all queries for which the system produces a verifiable, grounded response.
- **Abstention rate** – The fraction of queries where the system declines to answer due to weak, missing, or stale evidence.
- **Hallucination rate** – The fraction of outputs judged unsupported or factually incorrect based on vendor evidence.

These metrics are reported separately for:

- **Head vendors** (well-documented products),
- **Tail vendors** (sparse evidence), and
- **Noisy queries** (misspellings or ambiguous vendor names).

4. Results

Table 2: End-to-End Evaluation: Prompts, Outputs, Abstentions, and Evidence Quality

Prompt	Structured Accurate Result	Answer / Abstain	Reason
What is the latest version of React?	Version: 18.3.1 Release Date: 2024-05-15 Source: React official notes	Answer	High $s_{\max}$, $h > 0$, $\Delta < \tau_t$
What is the latest version of Redis?	Version: 7.2.4 Release Date: 2024-03-20 Source: Redis changelog	Answer	Strong retrieval + fresh evidence
What is the latest version of WP Job Openings?	**Abstain:** Insufficient fresh evidence	Abstain	$\Delta > \tau_t$ (stale)
What is the latest version of 1000projects.org?	**Abstain:** No reliable sources.	Abstain	$s_{\max} < \tau_s$, $h = 0$
What is the latest version of ABB Ability Edgenius?	Version: 3.4.2 Release: 2024-01-10	Answer	Strong retrieval and vendor match

5. Discussion

The end-to-end evaluation demonstrates that ReleaseGuard's abstention-first architecture effectively separates queries supported by strong, verifiable evidence from those where information is incomplete, stale, or inconsistent. Across all prompts, the system consistently avoided unsupported or hallucinated responses by conditioning generation on evidence relevance, coverage, and freshness.

For well-documented products such as *React* and *Redis*, ReleaseGuard produced correct and structured outputs. In these cases, high semantic similarity scores, non-zero live-evidence hits, and recent timestamps satisfied the abstention thresholds. The ability to combine historical retrieval with live OS feeds shows that ReleaseGuard grounds its answers in authoritative external sources rather than relying solely on parametric memory.

In contrast, queries such as *WP Job Openings* and *1000projects.org* triggered abstentions. These prompts exhibited low retrieval scores, absence of vendor-aligned context, or evidence with timestamps exceeding the freshness threshold. By refusing to answer, ReleaseGuard prevents fabricated version numbers or invented changelog entries,patterns commonly observed in baseline LLM outputs. Importantly, abstention occurred even when partial lexical matches existed, indicating that the classifier prioritizes evidence sufficiency over superficial similarity.

The case of *ABB Ability Edgenius* further demonstrates ReleaseGuard's robustness. Although retrieval density was lower than for React or Redis, the retrieved evidence was semantically aligned and sufficiently recent, leading the system to answer. This suggests that ReleaseGuard generalizes effectively across both popular and niche software products when unambiguous evidence is available.

Overall, the evaluation confirms the intended behavior of ReleaseGuard:

- **It answers only when evidence is strong, fresh, and vendor-relevant**, minimizing the likelihood of hallucinated update information.
- **It abstains when evidence is weak, missing, or stale**, preventing unsafe or misleading outputs.
- **It distinguishes genuine information sparsity from retrieval noise**, improving trustworthiness in update-intelligence scenarios.

These results show that abstention-first reasoning is not only capable of reducing hallucinations but is also *necessary* in safety-critical domains such as software-update intelligence. Incorrect version reporting, stale release data, or fabricated CVE information can propagate into CI/CD configurations, trigger failed deployments, or introduce operational instability. ReleaseGuard's proactive abstention mechanism provides a principled safeguard against these risks.

6. Conclusion

This study introduced ReleaseGuard, an abstention-first retrieval-augmented framework designed to reduce hallucinations in software-update intelligence tasks. By integrating query-intent filtering, hybrid retrieval, evidence-freshness analysis, and rule-based abstention, the system provides accurate and verifiable responses while avoiding unsupported claims. The end-to-end evaluation demonstrates that ReleaseGuard consistently answers only when the evidence is strong, recent, and vendor-aligned, and abstains when evidence is insufficient or unreliable. This behavior prevents common LLM failure modes such as fabricated version numbers, invented changelog details, and misleading update summaries.

The results confirm that abstention is an effective safeguard for update-oriented applications, where incorrect model outputs can cascade into operational issues, misconfigurations, or unsafe deployment decisions. By prioritizing correctness over coverage, ReleaseGuard shifts the design of update assistants toward reliability, transparency, and safety.

Overall, ReleaseGuard highlights the importance of evidence-aware mechanisms in RAG pipelines and provides a practical blueprint for building safer software-update intelligence systems. Future work will explore extending the abstention classifier with learned models, integrating richer vendor-specific signals, and evaluating the framework across larger, real-world deployment scenarios.

References

[1] Zhao, et al. (2023). *RAG still hallucinates with weak evidence.* arXiv preprint arXiv:2402.10612.

[2] Anonymous. (2023). *RAGTruth: Benchmarking hallucination under stale retrieval.* arXiv preprint arXiv:2401.00396.

[3] Luo, et al. (2024). *LLMs hallucinate when evidence is contradictory or outdated.* arXiv preprint arXiv:2508.03860.

[4] Kamalloo, Ehsan, et al. (2024). *RAGTruth: A benchmark for hallucination in retrieval-augmented language models.* arXiv preprint arXiv:2401.16421.

[5] Vaithilingam, R., et al. (2022). *Expectations vs. Experience: Evaluating GitHub Copilot's code generation.* CHI Conference Proceedings.

[6] Asai, Akari, et al. (2023). *Self-RAG: Learning to retrieve, generate, and critique through self-reflection.* arXiv preprint arXiv:2310.11511.

[7] Mirhosseini, A., & Parnin, C. (2017). *Can automated refactoring break your build?* Empirical Software Engineering.

[8] Ji, Z., et al. (2023). *Survey of Hallucination in Natural Language Generation.* Proceedings of ACL.

[9] Varshney, A., et al. (2023). *Investigating Hallucinations in LLMs.* arXiv preprint arXiv:2309.00068.

[10] Guu, Kelvin, et al. (2020). *REALM: Retrieval-Augmented Language Model Pre-Training.* Proceedings of ICML.

[11] Borgeaud, Sebastian, et al. (2022). *Improving Language Models by Retrieving from Trillions of Tokens.* Proceedings of ICML.

[12] Izacard, Gautier, et al. (2022). *ATLAS: Few-shot Learning with Retrieval.* NeurIPS Proceedings.

[13] Kadavath, Saurav, et al. (2022). *Language Models (Mostly) Know What They Don't Know.* arXiv preprint arXiv:2207.05221.

[14] Lin, T., et al. (2022). *Teaching Models to Abstain with AbstainQA.* Proceedings of ACL.

[15] Wei, Jason, et al. (2022). *Chain-of-Thought and Instruction Hierarchies.* NeurIPS Proceedings.

[16] Ganguli, D., et al. (2023). *Red Teaming LLMs to Understand Safety Risks.* arXiv preprint arXiv:2301.08650.

[17] Izacard, Gautier & Grave, Edouard. (2021). *FiD: Fusion-in-Decoder for Open-Domain QA.* ICLR Proceedings.

[18] Lin, et al. (2022). *Teaching Models to Express Uncertainty.* arXiv preprint.

[19] Bai, Y., et al. (2022). *Constitutional AI: Harmlessness from AI Feedback.* arXiv preprint.

[20] Kadner, et al. (2023). *IDK Training for Language Models.* Proceedings of ACL.

[21] Wang, Xuezhi, et al. (2023). *Self-Consistency Improves Chain-of-Thought Reasoning.* Proceedings of ICLR.

[22] Shuster, K., et al. (2021). *Retrieval Augmentation Reduces Hallucination.* Proceedings of ACL.

[23] Shi, Weijia, et al. (2023). *Large Language Model Verification.* Proceedings of NeurIPS.

[24] Mallen, A., et al. (2023). *Trust Improvements via Model Checking.* arXiv preprint.

[25] Taveekitworachai, Pittawat, Abdullah, Febri, & Thawonmas, Ruck. (2024). *Null-Shot Prompting: Rethinking Prompting Large Language Models with Hallucination.* EMNLP Proceedings, pp. 13321–13361.

[26] Madhusudhan, A., Sripada, S., Basu, R., Bhattacharya, S., & Chandu, K. (2025). *Do LLMs Know When to NOT Answer? Investigating Abstention Abilities of Large Language Models.* COLING Proceedings, pp. 7224–7244.

[27] Hoffmann, Rafael, Liu, Yuxin, Wang, Rohan, Zelle, Jens, Singh, Aditi, & Huang, Jason. (2025). *Abstention Policies for Automated Program Repair Agents.* arXiv preprint arXiv:2510.03217.

[28] Krishna, Keshav, et al. (2023). *RARR: Retrieval-Augmented Rewriting for Reliable Large Language Model Generation.* ICML Proceedings.

[29] Jang, Joel, et al. (2023). *FreshPrompt: Mitigating Hallucinations from Out-of-Date Evidence in Retrieval-Augmented Generation.* arXiv preprint arXiv:2310.03214.

[30] Chen, Wenhu, et al. (2024). *Program-of-Thoughts Verification: Accelerating LLM Reasoning with Precision.* NeurIPS Proceedings.

[31] Shi, Weijia, et al. (2023). *Large Language Model Verification.* NeurIPS Proceedings.

[32] Zou, Andy, et al. (2023). *SCAlign: Safety-Controlled Alignment via Selective Refusal Fine-Tuning.* arXiv preprint arXiv:2310.06282.